

The Commit Is the Label: Four Problems in Agent Memory That Version Control Already Answers

Frank Guo¹

¹Rekal — rekal.dev · guocongmit@gmail.com · github.com/rekal-dev/rekal-cli

Abstract. Memory systems for AI coding agents are rebuilding, in software, guarantees that version control already provides. An agent’s memory is context assembly under a token budget — for each question, the few thousand recorded tokens that change the answer — and the literature assembles it with machinery: tiered stores, memory graphs, and compiled wikis, guarded by model-judged admission and evaluated on hand-annotated benchmarks. The machinery imports four problems it then cannot discharge: **annotation** (no ground truth without human labels), **staleness** (compiled structure must be maintained), **self-confirmation** (a judge that can admit its own mistakes), and **contamination** (one shared pool, one open write path). In the software-engineering setting, each is answered by a git primitive every team already runs: the commit labels which sessions produced which verified change; rebuild and diff make derived structure disposable and its drift reviewable; the merge externally verifies what enters shared memory; and code review is the sole, audited channel across scope boundaries. These are answers **by construction** — properties of how the system is built, checkable by reading it. This paper puts the first to an empirical test: we build **Rekal**, a local-first, single-binary memory engine, and derive **RekalBench**, a self-labeling benchmark for repo-grounded intent recall whose ground truth is mined — not annotated — from the corpus’s own commit-session links. The other three we substantiate as design guarantees, not measured outcomes, and say so. On a real working corpus of 1,433 sessions and 66k turns, retrieval over the parsed, scrubbed ledger is categorically more effective than term-frequency scanning of the raw transcripts (0.187 vs 0.005 pooled MRR); against a strong lexical index over the same parsed turns (0.171) the hybrid’s gain is smaller and, as predicted, concentrated on paraphrased provenance queries, where the producing session is returned in the top five 54% of the time. Recall runs at ~0.5s median, and a bounded drill assembles answer-bearing context in ~1,500 tokens. The prediction was fixed before the held-out split was scored; the benchmark harness is public, fully local, and runnable by anyone on their own history at zero annotation cost.

1 Introduction

Code has a ledger; intent does not. Version control records every line a team ships, but the reasoning that produced those lines — explored designs, rejected alternatives, the correction a reviewer shouted mid-session — lives in AI-

assistant transcripts that expire with the terminal window. The cost compounds as agents do more of the work: an agent that cannot remember what its own team already tried will confidently re-propose it.

Start from what memory **is** for an agent. The context window is the scarce resource, so memory is not a store — it is **context assembly under a token budget**: for each question, deliver the few thousand tokens that change the answer, out of millions of recorded ones. That framing fixes the requirements. Recall must be **bounded** — a budget-shaped set the agent can drill, not an unbounded scan [1]. What it returns must be **fresh** — reflecting the repository as of the last commit, not a compaction of last month. Every memory must be **lineaged** — dereferenceable to the source turn and the commit it produced, because an unattributable memory cannot be trusted or audited. Anything **shared** must be verified by something other than the model’s own judgment. And the whole system must be **cheaply evaluable** — a memory whose quality cannot be measured without an annotation team will not be measured at all.

The research community meets these requirements with **machinery**: organize dialogue history into tiered stores [2], associative graphs [3], or compiled wikis [4]; guard the store with model-judged admission [5]; evaluate on hand-annotated conversational suites [6]. Each piece of machinery then becomes its own research problem, and we name the four: **staleness** — the compiled structure must be maintained; **self-confirmation** — the judge can admit its own mistakes; **annotation** — the evaluation does not scale; **contamination** — the shared pool becomes an attack surface [7]. A recent systematic evaluation from the data-management perspective — twelve memory systems, five workloads, eleven datasets — reaches the fitting verdict: **no single architecture dominates; effectiveness depends on aligning memory structure with the workload** [8].

This paper takes that verdict literally. For coding agents, the workload already has a structure — the repository — and **the four open problems the machinery addresses are already answered by infrastructure every software team runs** (Table 1). We are precise about what “answered” means, and it differs across the four. Annotation is answered **and demonstrated**: the commit-session link is a free label, and this paper builds a working retrieval benchmark on it. The other three —

Open problem	The literature’s machinery	The git-native answer (Rekal mechanism)
Annotation. Where does supervision come from?	Human annotation of dialogue corpora [6]; LLM judges grading themselves	The commit. A post-commit hook links sessions to the change they produced — free, objective, abundant (§4.1); RekalBench mines its gold from these links (§5)
Staleness. How does derived memory stay current?	Consolidation daemons; compiled wikis and graphs whose maintenance is “future work” [2], [4]	Rebuild + diff. Indexes are disposable and rebuilt (§4.2); materialized views regenerate deterministically, so drift arrives as a reviewable <code>git diff</code>
Self-confirmation. Who verifies what enters shared memory?	Self- or consensus-judged admission [5]	The merge. Only sessions whose commit landed on the default branch are shared; review + CI + merge are the external verifier (§4.3)
Contamination. How does private memory cross a boundary?	One machine-wide pool, implicitly readable and writable everywhere [2], [9] — the documented contamination surface [7], [10]	The review. Cross-repo memory is index-only and structurally unpushable; its sole egress is an origin-labeled page on a PR (§4.4)

Table 1: The thesis. Four problems the agent-memory literature treats as open research, and the git primitive that answers each by construction. §4 gives the mechanisms. Only the first — annotation — is put to an empirical test in this paper (§5–7); the other three are argued as design guarantees, not measured here.

staleness, self-confirmation, contamination — are answered **by construction**: they are properties of how Rekal is built (a disposable index, a merge-gated export path, a review-only egress), verifiable by reading the design, but we do not here measure their downstream benefit and do not claim to. The contribution is thus a memory system built **into** git, the self-labeling benchmark that construction makes possible, and a first retrieval result on it — not a claim to have empirically closed four problems at once.

Contributions. (1) The design and implementation of Rekal, a local-first, git-native memory engine for coding agents (single binary; capture, storage, hybrid recall, and team sync with no server, API, or telemetry), with an agent-facing skill layer that operationalizes active, multi-step memory reconstruction [3]. (2) **RekalBench**, a self-labeling benchmark methodology for repo-grounded intent recall, whose ground truth is mined from the corpus’s own commit-session structure rather than annotated, together with a first instantiation on two task families. (3) A retrieval evaluation on a real working corpus, with single-signal ablations (free under query-time weighting) and a judge-free context-assembly measure, whose one prediction was fixed before the held-out split was scored; the judged-answer and corpus-scale rungs are specified for the same public harness but not run here, and we say so plainly rather than implying a completeness the study does not have.

2 A Worked Example

Everything in this paper appears once, concretely, in the following five-minute story. (Illustrative; the exact JSON shapes are the shipped ones.)

On Monday, an engineer and an agent rework webhook delivery. Mid-session the engineer interrupts: **“don’t retry on a fixed delay — it stampedes the downstream on recovery.”** The agent switches to exponential backoff with jitter; the change merges. The post-commit hook captures the session — turns, tool calls, the interruption tagged with its own role (`human_steering`) — scrubs secrets, and

appends it to the repo’s ledger with a link to the commit SHA. Nobody wrote documentation.

On Friday, a different agent, different machine, same team, picks up a related task and asks:

```
$ rekall "should webhook retries use a fixed delay?"
{ "results": [{
  "session_id": "01JNQX8F2K9M",
  "score": 0.87,
  "snippet": "don't retry on a fixed delay - it
    stampedes the downstream on recovery",
  "snippet_turn_index": 41,
  "snippet_role": "human_steering",
  "summary_turn_index": 118,
  "session": { "commit": "9c2f41a",
    "files": ["delivery/retry.go", ...] }
}]}
```

Four properties of the answer carry the paper’s argument. The top snippet is the **steering turn itself** — ranking boosts the moments a human redirected the agent, because that is where decisions actually get made. `snippet_role` says a human said this; machine text can never masquerade as intent. The result is a **pointer structure**, not a payload: `snippet_turn_index` locates the evidence, `summary_turn_index` points at a harness-written distillation of the whole session (10–17KB, already paid for during context compaction — never inlined), and the agent drills exactly as deep as the question warrants. And the `commit` field is the provenance anchor: the claim “we ruled this out” dereferences to a change that survived review and merge. The dead end was re-proposed on Friday and ruled out in one query — by a colleague’s Monday, not by documentation.

The rest of the paper is this example generalized: §3 the engine that produced it, §4 the four guarantees behind its fields, §5 how a corpus of such sessions labels itself, §6–7 the measurements.

3 The Engine

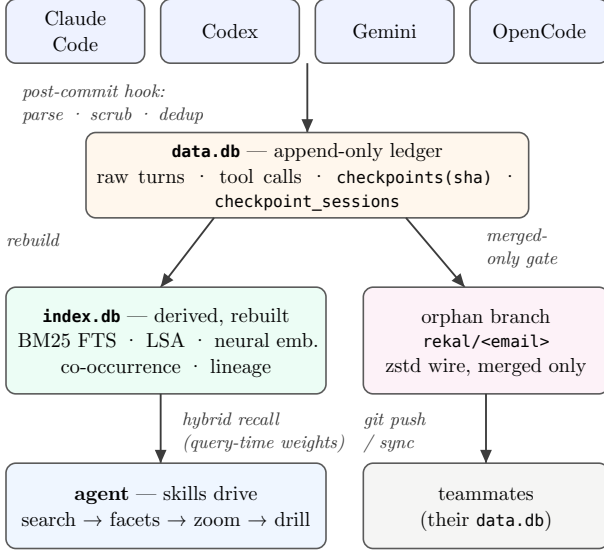


Figure 1: Rekal architecture. One source of truth (append-only `data.db`); all derived structure disposable (`index.db`); sharing gated on merge verification. No server, no API, no telemetry — git is the only wire.

Rekal is one binary embedding its database engine, embedding models, and compression dictionary. A post-commit hook parses the active assistant session(s) — adapters cover Claude Code, Codex, Gemini, and OpenCode — deduplicates turns, scrubs secrets and anonymizes paths **before any byte is stored** (the ordering shown to be the only safe one [10]), and appends to `data.db`. Nothing is summarized at write time: following the preservation result of [11], the unit of storage is the raw turn with role, order, timestamp, and tool-call structure, and every recall result carries source-turn attribution. Write-time compression is lossy and unfixable; query-time distillation improves with every model generation.

One class of distillation is nonetheless **harvested**: when the harness compacts a long session it writes an LLM summary of the conversation so far back into the transcript. Rekal captures these as turns with a dedicated role **summary** — already paid for, cumulative (the latest subsumes the rest), and never confused with human intent. Rekal generates no summaries of its own: a commit-time summary would be query-blind, written before any question exists, whereas the drill is query-aware and paid lazily.

Recall scores sessions by a weighted hybrid of BM25, latent-semantic, and neural similarity; boosts **human_steering** turns (the example’s Monday interruption) and, by a smaller factor, harvested summaries; down-weights sub-agent transcripts; and returns the pointer-structured JSON of §2. Weights live in local config and apply at **query time** — no reindex, which is also what makes the single-signal ablations of §7 free. In RISE’s terms [1], search constructs a **bounded interaction space** and the drill

tools explore it; a layer of six shipped skills (base search, provenance, self-reflection, distillation, exhaustive census, wiki materialization) is the active-reconstruction policy [3] that sequences those primitives.

4 Four Guarantees from Git

The engine is ordinary; the guarantees are not. This section walks Table 1 row by row, confronting in each row the machinery it replaces.

4.1 Annotation: the commit is the label

Every checkpoint records which sessions produced which commit (checkpoint_sessions). This single join is the paper’s namesake and its most consequential design choice: it connects a unit of **intent** (the session) to a verified unit of **code** (the commit) — automatically, at zero marginal cost, thousands of times per repository-year. Dialogue-memory research must manufacture this connection by human annotation [6] or accept LLM judges grading their own homework; here the connection is a side effect of using version control. Everything downstream leans on it: provenance answers in recall (§2), the merged-only sharing gate (§4.3), and the entire benchmark (§5), whose five task families are all mined from this one link plus git topology.

4.2 Staleness: rebuild and diff

`data.db` is the only source of truth and is append-only. `index.db` holds everything derived — full-text, embeddings, co-occurrence, lineage, facets — and is **disposable by construction**: any migration, corruption, or model upgrade is handled by deletion and rebuild (content-hash-keyed embedding caching keeps rebuilds incremental). This is the structural answer to the maintenance problem that compiled-memory systems name and defer [2], [4], [12]: **the freshness of a structure you can throw away is not a research problem**.

The same discipline extends to memory the team can browse. A wiki skill materializes topic pages (`docs/wiki/<topic>.md` plus a graph-mapped index) from file co-occurrence clusters and the sessions behind them, shipped as an ordinary pull request. Generation is **deterministic** — topics and edges sorted, thresholds fixed — so a re-run over unchanged history is an empty diff, and any non-empty diff is structural drift: an edge appearing in the topic graph is a correlation that did not exist last run; a removed edge is one that decayed; each transition is reviewed before admission. Mutating graph stores cannot show this drift to anyone; here **git log** over the index is a reviewed time series of the project’s conceptual structure. The maintenance problem is not solved — it is **converted into code review**, a process teams already run. Because pages are a cache of memory, not the memory, whether that cache pays for itself is an empirical question this paper does not settle: the generation cost is recoverable from the ledger’s own lineage records and the payoff is measurable against recall+drill, but we report the mechanism here and leave the economics to the harness.

4.3 Self-confirmation: the merge is the gate

rekal push exports to a per-author git orphan branch only those checkpoints whose commit is reachable from the default branch (with patch-equivalence detection for squash merges); everything else waits, releasing automatically if its branch later lands. The shared tier therefore admits only experience that survived review, CI, and merge — an **external** verification gate, in contrast to the self- or consensus-judged admission proposed to escape the self-confirmation trap [5]: when the same model family executes, summarizes, and admits its own memories, wrong-but-self-consistent trajectories are stored as successes and amplified on reuse. The merge gate has a second, subtler yield: unmerged and abandoned branches remain in the local ledger as labeled **negative** knowledge — the map of dead ends — and §5’s task T3 tests exactly whether a system can warn “we tried this; it didn’t land.”

4.4 Contamination: review as the egress channel

Rekal’s memory has three scopes with asymmetric permeability. The **repo ledger** is truth. The **machine-wide index** optionally folds in the operator’s other repos’ sessions (explicit opt-in) — index-only, origin-labeled, and structurally unpushable: an imported session has no checkpoint in this repo’s ledger, so no code path can export it. The **team wire** admits merged work only. Knowledge crosses a scope boundary exclusively through a human-visible artifact: sessions reach the team when their commit merges, and cross-repo experience becomes committed text through exactly one channel — a wiki page generated in an explicit cross-repo mode that labels every foreign citation with its origin, shipped as a PR whose body declares what is crossing. Machine-wide stores in the literature make the opposite choice [2], [9]: one pool, implicitly readable and writable everywhere — precisely the open write path the security analyses identify as the contamination and exfiltration surface [7], [10], [13], and controlled experiments show sanitization only works **before** summarization, which is why Rekal scrubs before insertion and stores raw. Governance concerns for enterprise memory [14] map onto git’s existing controls. The rule compresses to: **wide reads locally, narrow writes globally — egress is always a diff someone approved.**

5 RekalBench

No benchmark exists for repo-grounded intent recall: conversational-memory suites (LoCoMo [6], Long-MemEval, PERSONAMEM) evaluate chat-persona recall, and IR suites (BEIR, BRIGHT) evaluate document QA. RekalBench’s defining property follows from §4.1: **the corpus labels itself.**

Watch one label being born. The Monday session of §2 checkpointed against commit **9c2f41a**. The miner (plain SQL over the ledger, plus git) emits, with no human in the loop:

```
{ "task": "t1",
  "gold": ["01JNQX8F2K9M"],
  "commit": "9c2f41a",
  "source": { "subject": "webhooks: switch retries
              to exponential backoff",
              "files": ["delivery/retry.go", ...] } }
```

An LLM paraphrases the source material into a natural developer question (“how did we end up handling webhook retry timing?”), a 4-gram Jaccard ceiling (≤ 0.30 , one aggressive-paraphrase retry, else the label is dropped and logged) breaks lexical leakage between query and target, and the pair joins the query set. The gold is objective — that session verifiably produced that commit — and the supervision cost is zero. The same construction yields all five task families (Table 2); the steering turn of §2 is likewise a T2 gold, and the never-merged branches of §4.3 are T3 gold **because git says so.**

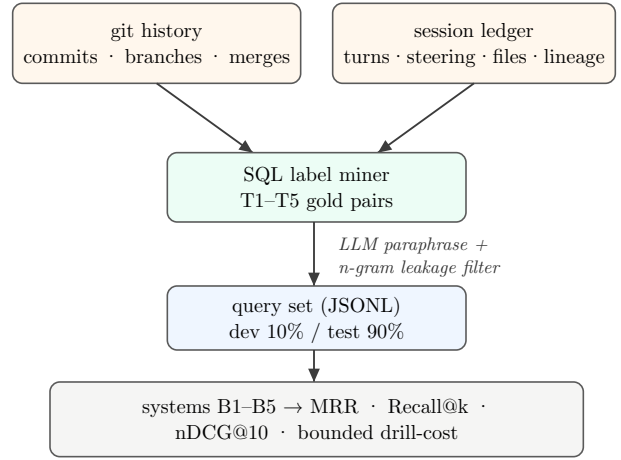


Figure 2: RekalBench pipeline. Labels are mined from structure the tool already records; paraphrase plus an n-gram overlap ceiling breaks lexical leakage between query and target.

5.1 Tasks

Task	Gold label source	Query generation	n
T1 Provenance	commit $\rightarrow$ producing session(s), via the checkpoint_sessions links	paraphrase of commit message + changed paths (not indexed content)	152
T2 Decision recall	human_steering turns	paraphrase of surrounding context; steering turn held out of prompt; 4-gram Jaccard ≤ 0.3	263
T3 Dead-end awareness	sessions on never-merged branches	“have we tried X?” from the branch’s cumulative intent	0 [†]
T4 Multi-hop synthesis	session pairs linked by file co-occurrence / lineage	generator-validated two-session questions	—
T5 Decision drift	later session reversing an earlier decision on the same files (hand-confirmed sample)	“what is our current approach to X?”	—

Table 2: RekalBench task families, all mined from recorded structure. This study evaluates T1 and T2 — the families this corpus labels at scale (415 pairs). T3–T5 are defined by the same construction and evaluated wherever a corpus supplies their structure: [†]this corpus is effectively single-branch (four branches, all merged), so it yields no dead-end labels; T4/T5 (the latter motivated by the belief-revision result of [15] — memory must surface the **current** decision) await a repo with the requisite topology and run on the same public harness.

Label noise. A commit’s linked sessions can include incidental chatter; because the label is the objective commit-session link rather than a subjective judgment, this noise lowers measured scores (a hit on chatter misses the gold turn) rather than inflating them, so the retrieval numbers are conservative. To keep the query independent of the target, T1 queries are paraphrased from the commit message and changed paths — content the index never sees — under a 4-gram Jaccard ceiling that drops leaky pairs. **Splits.** 10% dev (tuning allowed) / 90% test (35/380 after the leakage filter), one-shot — the incumbent-versus-candidate discipline of [16].

5.2 Corpus and systems

One operator’s real working store, folded per-repo (labels require the repo’s own checkpoint ledger) plus machine-wide for scale sweeps. The primary repo holds 1,433 sessions, 66,323 turns, and 88,144 tool calls, with 1,219 steering turns and 432 harvested compaction summaries across 152 commits carrying linked sessions (1,060 commit-

session links), spanning 2026-03-22 to 2026-07-10; the machine-wide fold adds 3,618 imported sessions from 18 other projects for the scale sweeps. No session content leaves the machine; the paper reports aggregates only.

ID	System
B1	Grep-rank over raw transcript JSONL — the non-agentic direct-corpus-interaction proxy [17]: sessions ranked by term-frequency hits, no parsing or indexing
B3	BM25-only (Rekal weights {1,0,0}) — query-time ablation, no reindex
B4	Neural-only (weights {0,0,1}) — ablation
B5	Rekal full hybrid + steering boost + summary boost + subagent down-weight

Table 3: Systems under test. B1 is the honest lexical baseline for retrieval: it is what “just grep the transcripts” scores when the transcripts are raw. The agentic form of direct corpus interaction — a model driving `rg/jq` over a turn budget [17], [18] — is a judged-rung comparison the same public harness supports. B3/B4 are free query-time ablations of B5.

Metrics. Retrieval is scored by MRR, Recall@{1, 5, 10}, and nDCG@10 with 1,000-resample bootstrap CIs, on the held-out test split. Beyond retrieval, the same self-labeled query set drives a bounded context-assembly measure — the tokens a drill ingests to cover the gold turn’s distinctive content, with no LLM in the loop — reported in §7. Judged answer quality (distinct generate/answer/judge models against the gold turn) and a corpus-scale sweep are the natural extensions the public harness enables at zero further labeling cost.

6 Prediction and Protocol

The four guarantees of Table 1 divide by **how they are verified**. Self-confirmation and contamination are answered **by construction**: the merge gate and the egress restriction are code paths, checkable by reading them — an imported session has no checkpoint, so no export path exists, and no experiment can add to that assurance. Annotation is answered by the benchmark’s very existence: the corpus labels itself (§5), so the supervision cost that dialogue-memory work must pay is simply not incurred. What remains genuinely open — the one thing a reader should demand evidence for — is **retrievability**: does memory built on these guarantees actually surface the right prior session better than the alternative of not building it at all? A system whose recall loses to grep needs no philosophy.

We fixed one prediction, and a mechanistic expectation about **where** it would hold, before scoring the held-out test split (the source commit predates the results in the project’s own history):

Prediction (retrievability). On the held-out test split, Rekal’s hybrid recall (B5) beats term-frequency grep over the raw transcripts (B1) on pooled MRR and nDCG@10 with non-overlapping bootstrap intervals.

System	Pooled MRR (95% CI)	nDCG@10	T1 MRR	T1 R@5	T2 MRR
B1 grep-rank (raw)	0.005 [.001,.011]	0.004	0.014	0.015	0.000
B4 neural-only	0.057 [.040,.078]	0.059	0.130	0.194	0.018
B3 BM25-only	0.171 [.145,.202]	0.219	0.248	0.425	0.130
B5 Rekal hybrid	0.187 [.158,.217]	0.227	0.301	0.537	0.124

Table 4: Retrieval quality on the held-out test split (n=380; pooled MRR with bootstrap 95% CIs, other columns point estimates). The registered prediction holds — hybrid recall beats raw-transcript grep with non-overlapping intervals — but that comparison only establishes a floor: grep-rank over raw JSONL is near-zero because the transcripts are adversarial material, so the informative contrast is against the strong lexical index over the **same parsed turns** (B3). There the pooled gap is small and its CIs overlap (0.187 vs 0.171); the hybrid’s advantage is not uniform but **located** — it is a provenance effect (T1 MRR 0.301 vs 0.248, R@5 0.537 vs 0.425) and vanishes on decision recall, where lexical is if anything ahead (T2 0.124 vs 0.130). The semantic layer earns its place exactly where paraphrase opens a surface-form gap, and not elsewhere.

Expectation (signal composition). The semantic layer earns its keep specifically on paraphrased provenance queries (T1), where the question shares little surface form with its target; on decision recall (T2), where a human’s steering words tend to recur, lexical matching already suffices.

The protocol follows the incumbent-versus-candidate discipline of [16]: weights may be tuned on the 10% dev split; the 90% test split is scored once.

7 Results

Reading the table. First, the floor. The gap between any Rekal configuration and raw grep is categorical, not marginal: even BM25-only over parsed, scrubbed turns (0.171) outscores term-frequency grep over the raw JSONL (0.005) by thirty-fold, because raw transcripts are adversarial retrieval material — tool dumps, base64 blobs, duplicated sidechains — and the index is not. We read this not as a hard baseline defeated but as the paper’s structural claim made measurable: the value is in treating the ledger as parsed memory, not as text to scan; a skilled agent driving grep would do better, and that agentic comparison is the judged rung we leave to the harness. Second, and more honestly, the hybrid’s advantage over strong lexical search is **modest and located**. Pooled, B5 and B3 are within each other’s confidence intervals; the separation is real only on provenance queries, exactly where the registered expectation put it. This is the alignment-by-construction premise in miniature: on a single-repo corpus, question and session share vocabulary, so lexical carries most of the signal and the neural layer pays off precisely when a paraphrase widens the surface-form gap. In absolute terms the numbers are moderate — a pooled MRR of 0.187 puts the first relevant session around rank five — but for the provenance questions the tool is built to answer, the producing session is in the top five 54% of the time, at a median recall latency of ~0.5s.

Once recall reaches the gold session, a five-turn window around the matched turn assembles the answer-bearing context — covering roughly 69% of the gold turn’s distinctive content words — in about 1,500 tokens, two to three orders of magnitude below dumping the session (10–17KB)

or the transcript. The bounded interaction space [1] is what keeps the token budget small: recall narrows to a handful of sessions, the drill reads only the neighborhood that matters.

8 Positioning

The design confrontations live inline in §4; what remains is where Rekal sits in the field’s own coordinate systems.

On the four-module anatomy of the data-management evaluation [8] — representation & storage, extraction, retrieval & routing, maintenance — Rekal classifies as a multi-paradigm hybrid on the first three: heterogeneous multi-engine storage (relational + full-text + vector in one embedded database), **deterministic** extraction (parsing and harvesting; uniquely among the taxonomy’s systems, no LLM sits anywhere in the write path — the property the contamination analyses require), and multi-stage hybrid retrieval with agentic routing via the skill layer. On the fourth module it exits the taxonomy’s design space entirely: that survey’s maintenance column enumerates LLM-driven semantic consolidation, capacity-driven eviction, and timestamp multi-versioning — each a paid, lossy, or complexity-bearing liability — whereas Rekal’s maintenance is **rebuild, diff, and review**: nothing is consolidated (append-only), nothing is evicted (raw is cheap), and versioning is git itself. The same evaluation’s no-free-lunch finding — effectiveness depends on aligning memory structure with the workload — is this paper’s premise taken to its limit: the coding workload’s structure is the repository, and Rekal aligns by construction rather than by tuning. On the **compression axis** — aggressive consolidation [2] versus preservation with attribution [11] — Rekal takes EMem’s side at the storage layer (raw turns, attributed snippets) and harvests, rather than generates, distillation. On the **retrieval axis** — one-shot lookup versus active reconstruction [3] — it takes MRAgent’s side at the interaction layer: the skill playbooks drive iterative search–facet–zoom–drill loops over engine primitives, and LLM-Wiki’s page-traversal result [4] is recovered, without the compilation liability, as the reviewed wiki of §4.2. Architecturally the storage is aligned with SAG’s query-time joins over flat indexes [12]. Against retrieval-free direct

corpus interaction [17], [18] the position matches RISE [1]: a bounded interaction space beats an unbounded scan as the corpus grows, and our result is the sharp form of that argument at the retrieval layer — term-frequency scanning of raw transcripts is not competitive with recall over parsed memory even on one developer’s history; mapping the full accuracy-versus-scale crossover against an **agentic** grep baseline is the natural next rung, on the same public harness. Self-improvement from traces [9], [16], [19] is the flywheel this corpus natively enables (§9); LoCoMo-style persona memory [6] is out of scope by design.

9 Limitations and Future Work

Retrievability is a proxy: a session that ranks first is a session the agent **can** reach, not proof that the answer it then reads is correct. The bounded result is honest about its scope — it measures whether the right memory is findable and how few tokens a drill needs to cover it, not end-to-end task success. The natural next rung is judged answer quality against the gold turn, with distinct generate/answer/judge models; the query set and the gold labels already exist, so it adds a judge, not a labeling effort.

A single-operator corpus is a case study until replicated. The mitigation is structural rather than promissory: because the labels are mined, not annotated, the entire benchmark is public, fully local, and runnable by any Rekal user on their own store at zero marginal cost — replication does not wait on us. The same openness covers the task families this corpus cannot exercise (dead-ends need unmerged branches; drift needs a reversal history) and the scale sweep against an agentic-grep baseline.

Finally, the neural layer is the weakest signal here (B4, Table 4). That is consistent with the paper’s thesis — a single repo’s question and its answering session share vocabulary — but it also invites a direct test: substituting a stronger code-tuned embedding model, served over the HTTP backend (OpenAI-compatible or Amazon Bedrock), reindexing, and swapping back for free via the content-hash cache. A material lift to the **hybrid** would raise the shipped default; a null lift would be further evidence that alignment, not embedding quality, is the binding constraint [8].

10 The Flywheel, and What This Buys

The corpus contains its own improvement signal: which recalled sessions an agent drills into after a search is an implicit relevance label [19], enabling private per-corpus weight tuning accepted only on head-to-head wins over the incumbent configuration [16]; trajectory review can revise the skill layer itself [9]; and the wiki’s generation runs are themselves ledgered sessions, so even the distillation loop is self-accounting. None of this requires new machinery — it is the same ledger, read again.

Memory for coding agents does not need another compiled structure to maintain or another self-judged store to poison itself. It needs a ledger that already has ground truth. Git

supplies the label (the commit), the freshness mechanism (rebuild and diff), the verifier (the merge), and the egress channel (the review); Rekal supplies the capture, the disposable indexes, the bounded recall, and the playbooks. Three of those four answers we make by construction and leave for a reader to verify in the design; the fourth — the label — we put to work, and it yields a benchmark that labels itself and a retrieval result whose one firm lesson is that parsed, provenance-linked memory is findable where raw transcripts are not, with a semantic layer that earns its place on paraphrased questions. Because the labels are mined, not annotated, every part of that is falsifiable by anyone, locally, on their own history.

Reproducibility. Engine, skills, benchmark spec, extraction SQL, runbook (DATA-RUN.md), and this paper’s source: github.com/rekal-dev/rekal-cli (docs/research/). All benchmark data remains on the operator’s machine; published artifacts are aggregates, prompts, and code.

References

- [1] S. Zhuang and others, “Towards Retrieving Interaction Spaces for Agentic Search.” [Online]. Available: <https://arxiv.org/abs/2606.06880>
- [2] S. Yan, J. Ni, L. Zheng, and others, “AdaMem: Adaptive User-Centric Memory for Long-Horizon Dialogue Agents.” [Online]. Available: <https://arxiv.org/abs/2603.16496>
- [3] S. Ji, Y. Li, and B. Hooi, “Memory is Reconstructed, Not Retrieved: Graph Memory for LLM Agents.” [Online]. Available: <https://arxiv.org/abs/2606.06036>
- [4] H. Ming, F. Li, X. Wu, W. Que, and others, “Retrieval as Reasoning: Self-Evolving Agent-Native Retrieval via LLM-Wiki.” [Online]. Available: <https://arxiv.org/abs/2605.25480>
- [5] S. Zhu, Y. Qi, Y. Wang, and others, “Escaping the Self-Confirmation Trap: An Execute-Distill-Verify Paradigm for Agentic Experience Learning.” [Online]. Available: <https://arxiv.org/abs/2606.24428>
- [6] A. Maharana and others, “Evaluating Very Long-Term Conversational Memory of LLM Agents.” [Online]. Available: <https://arxiv.org/abs/2402.17753>
- [7] Survey authors, “A Survey on Long-Term Memory Security in LLM Agents: Attacks, Defenses, and Governance Across the Memory Lifecycle.” [Online]. Available: <https://arxiv.org/abs/2604.16548>
- [8] W. Zhou and others, “Are We Ready For An Agent-Native Memory System?.” [Online]. Available: <https://arxiv.org/abs/2606.24775>
- [9] S. Wu, H. Zhu, Y. Zhang, X. Wang, and S. Yeung-Levy, “AutoMem: Automated Learning of Memory as

a Cognitive Skill.” [Online]. Available: <https://arxiv.org/abs/2607.01224>

- [10] State-contamination authors, “State Contamination in Memory-Augmented LLM Agents.” [Online]. Available: <https://arxiv.org/abs/2605.16746>
- [11] S. Zhou and J. Han, “A Simple Yet Strong Baseline for Long-Term Conversational Memory of LLM Agents.” [Online]. Available: <https://arxiv.org/abs/2511.17208>
- [12] Y. Wu, J. Li, X. Liang, and others, “SAG: SQL-Retrieval Augmented Generation with Query-Time Dynamic Hyperedges.” [Online]. Available: <https://arxiv.org/abs/2606.15971>
- [13] Survey authors, “From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents.” [Online]. Available: <https://arxiv.org/abs/2606.04990>
- [14] H. Taheri, “Governed Memory: A Production Architecture for Multi-Agent Workflows.” [Online]. Available: <https://arxiv.org/abs/2603.17787>
- [15] BeliefShift authors, “BeliefShift: Benchmarking Temporal Belief Consistency and Opinion Drift in LLM Agents.” [Online]. Available: <https://arxiv.org/abs/2603.23848>
- [16] W. Pan and others, “Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference.” [Online]. Available: <https://arxiv.org/abs/2606.05922>
- [17] Z. Li, H. Zhang, and others, “Beyond Semantic Similarity: Rethinking Retrieval for Agentic Search via Direct Corpus Interaction.” [Online]. Available: <https://arxiv.org/abs/2605.05242>
- [18] GrepSeek authors, “Training Search Agents for Direct Corpus Interaction.” [Online]. Available: <https://arxiv.org/abs/2605.29307>
- [19] Y. Zhou and others, “Learning to Retrieve from Agent Trajectories.” [Online]. Available: <https://arxiv.org/abs/2604.04949>